

Computing for Analytical Work

Session 2 — What is running?

Block 3 — Python interpreters, packages, imports, and environments

Knowledge check 2 — 10 minutes, closed book, 3 questions

- Covers Session 1 and Homework 1: paths, working directory, terminal, `git status`
- Laptops closed. Paper only.
- Three short questions. Diagnostic, not tricky.
- Worth 5% of the final grade
- **This block's lecture is 35 minutes**, not 45 — the check comes out of lecture, never out of lab
- Hand it in when the timer goes. Then we begin.

This block in one sentence

"Python" means a particular interpreter with a particular set of installed packages.

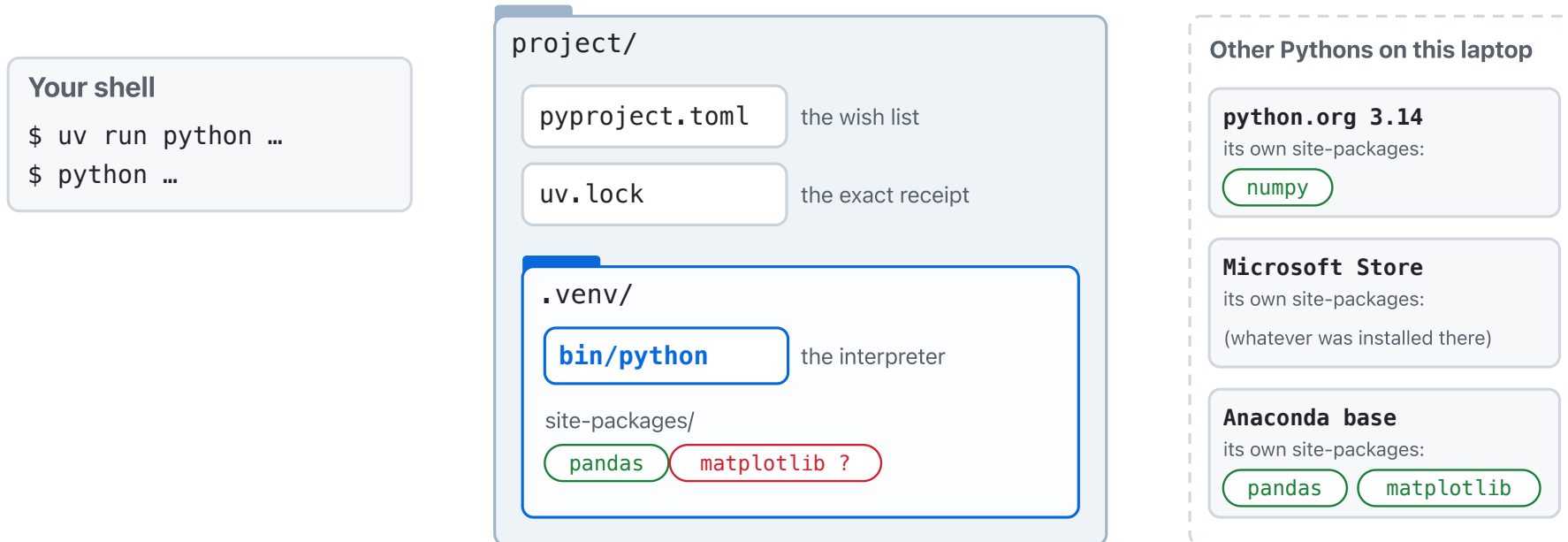
Not a language. Not an icon. A specific program, on a specific path, with a specific list of packages it can see.

What we cover

- The reveal: what `uv run` has been doing all week
- Interpreter, script, notebook
- Packages, `import`, and `sys.path`
- Installing into a specific environment; why installs "fail"
- Many Pythons on one machine, and how to check which is active
- `uv` : the one workflow this program uses
- Git thread: `git add`, `git commit`
- AI thread, stretch, and Lab 3

You have been typing `uv run` for a week

`uv run python scripts/summarize.py` — in the setup check, Lab 1, Lab 2, Homework 1. Every README said: *"You will learn what `uv run` does in Session 2. For now, type it exactly."* It is Session 2. This map is the answer.



Get the block's project: clone Lab 3 now

In your terminal (Git Bash on Windows, Terminal on macOS), in the folder where you keep course work:

```
git clone https://github.com/earino/ecbs5293-lab03-environments.git  
cd ecbs5293-lab03-environments
```

- Every command you *type* in this block is typed **there**: that terminal, that folder. Slides you only watch say so.
- It is the Lab 3 starter. Leave its README and its script alone until the lab; the lecture only looks at the project's environment.

Interpreter, script, notebook

- **Interpreter** — a program, one executable file, that reads Python code and runs it
- **Script** — a `.py` file the interpreter reads top to bottom, once, then exits
- **Notebook** — a long-running interpreter (the *kernel*) you feed one cell at a time

In that terminal, in the Lab 3 folder:

```
uv run python -c "import sys; print(sys.executable)"
```

That prints the path of the interpreter that ran the line. It is a file. You can `ls` it.

What a package is

- A folder of Python code somebody else wrote, plus metadata
- `pandas`, `matplotlib`, `jupyter` — each is a folder on disk, installed *somewhere*
- Installed means: copied into a folder **that one interpreter knows to look in**

Same terminal, same folder:

```
uv run python -c "import pandas; print(pandas.__file__)"
```

That prints where pandas lives. Note the path. It will matter in five slides.

What `import` actually does

```
import pandas
```

1. The interpreter takes a list of folders called `sys.path`
2. It walks that list, in order, looking for a folder or file named `pandas`
3. First match wins. No match: `ModuleNotFoundError`

`import` is a **search**. It is not a download, and it is not a question about your laptop.

`sys.path` belongs to *that* interpreter

`import pandas` is a search through one interpreter's `sys.path`, in order. First match wins.

sys.path of `.venv/bin/python`

- 1 `project/`
- 2 `.venv/lib/python3.13/site-packages` pandas ✓

found: stop, import it

sys.path of `/usr/bin/python3`

- 1 `/usr/lib/python3.9/`
- 2 `/usr/lib/python3.9/site-packages/`
- 3 `/usr/lib/python3/dist-packages/` no pandas

end of the list: `ModuleNotFoundError`

Same code. Two interpreters. Each searched its own list and told the truth.

So "is pandas installed?" is an incomplete question. **Installed for which interpreter?**

Installing into a specific environment

- An **environment** is one interpreter plus the folders it searches. That is all it is.
- Installing a package puts it into **one** environment
- This course: every project has its own environment, in `.venv/`, inside the project folder

```
uv add openpyxl
```

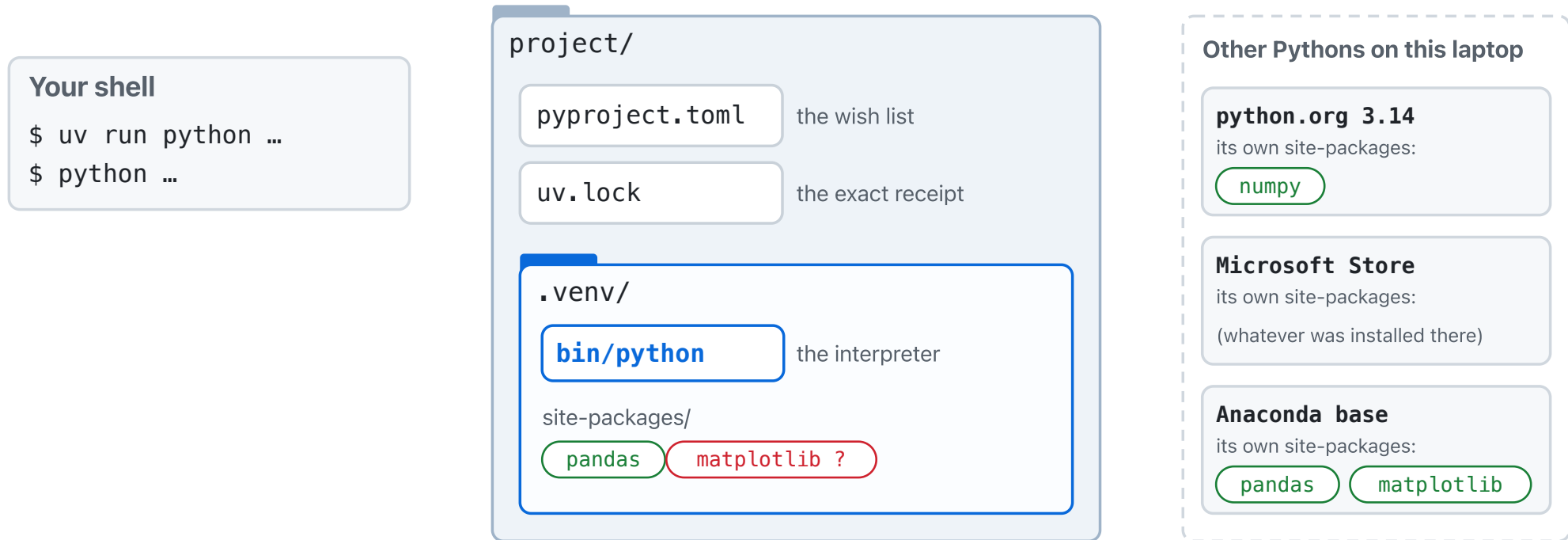
That installs openpyxl into *this project's* environment, and records that it did. **Watch; you type `uv add` in the lab.**

Why installs "fail"

```
Using cached six-1.17.0-py2.py3-none-any.whl (11 kB)
Installing collected packages: six, pyparsing, pillow, packaging, numpy, kiwisolver, fonttools, cyclor, python-dateutil, contourpy, matplotlib
Successfully installed contourpy-1.3.3 cyclor-0.12.1 fonttools-4.64.0 kiwisolver-1.5.1 matplotlib-3.11.1 numpy-2.5.3 packaging-26.3 pillow-12.3.0 pyparsing-3.3.2 python-dateutil-2.9.0.post0 six-1.17.0
(other) anna@laptop project $ uv run python -c "import matplotlib"
warning: `VIRTUAL_ENV=/Users/Shared/anna/other` does not match the project environment path `.venv` and will be ignored; use `--active` to target the active environment instead
Traceback (most recent call last):
  File "<string>", line 1, in <module>
    import matplotlib
ModuleNotFoundError: No module named 'matplotlib'
(other) anna@laptop project $
```

The install did not fail. It **succeeded, somewhere else**. A receipt from one Python, an error from another, one minute apart.

Many Pythons on one machine



Each has its own `sys.path`. Nothing coordinates them. **Which one runs is decided by which one you invoke.**

How the shell finds `python`: `PATH`

- `PATH` is a list of folders. The shell walks it in order; the first folder holding a `python` wins.
- `which python` prints that winner. Installing Anaconda, Homebrew, or the Store version puts a folder at the *front* of the list.
- `uv run` does not consult `PATH` for the interpreter. It goes to the project's `.venv/`.

Same idiom as `import`: a list, walked in order, first match wins.

How to check which Python is active

In your terminal, in the Lab 3 folder. All five only *look*; none of them changes anything:

```
python --version          # which Python is on PATH right now
which python              # where that Python actually lives
uv --version              # confirm uv is installed
uv python list            # which Pythons uv knows about
uv run python -c "import sys; print(sys.executable)" # the one inside the project env
```

Every one of these passes an argument to `python`. **Never type bare `python` in Git Bash:** on some setups it hangs with no error; on others it drops you into Python's `>>>` prompt. Neither is where you want to be.

Predict: two Pythons, side by side

```
python -c "import sys; print(sys.executable)"  
uv run python -c "import sys; print(sys.executable)"
```

My laptop. A terminal *outside* VS Code, standing in a project folder like yours, thirty seconds apart. **Will these two commands print the same path?**

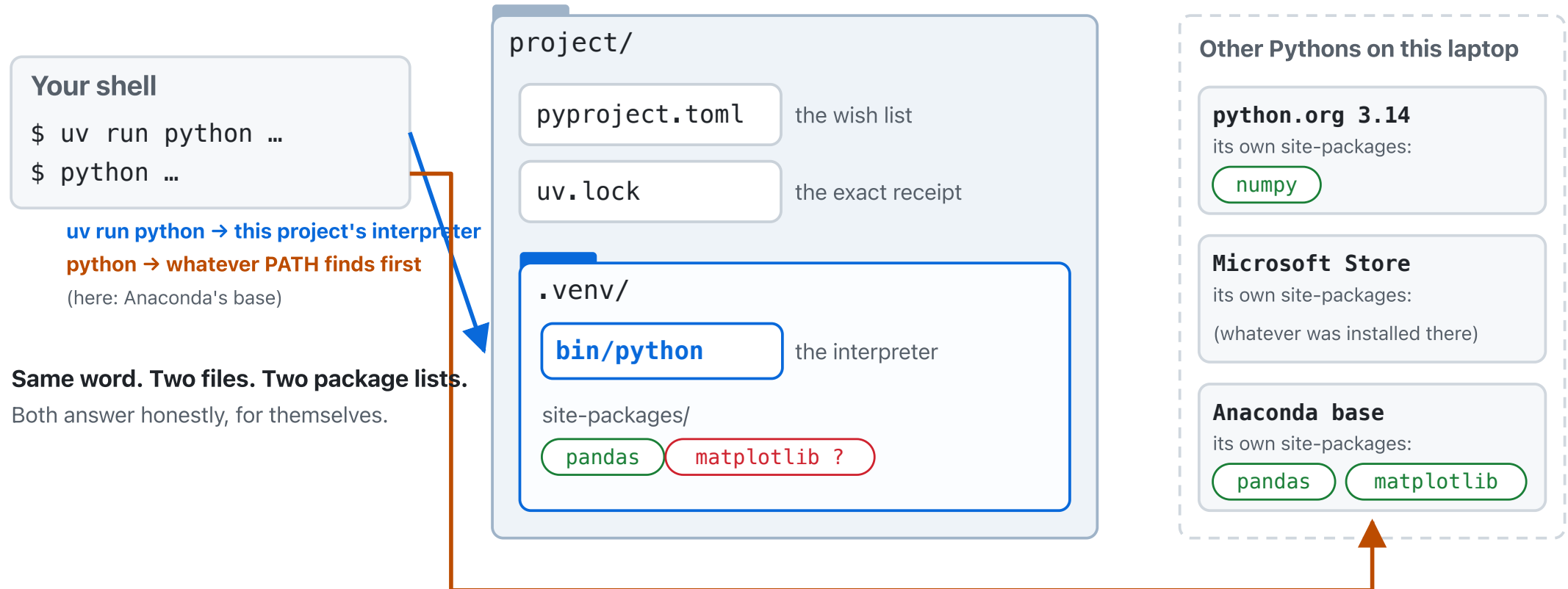
Write down *yes* or *no*, and one reason. Do not type yet.

Two Pythons — what happened

```
anna@laptop project $ python -c "import sys; print(sys.executable)"
/opt/homebrew/Caskroom/miniconda/base/bin/python
anna@laptop project $ uv run python -c "import sys; print(sys.executable)"
/Users/Shared/anna/project/.venv/bin/python3
anna@laptop project $ python -c "import matplotlib; print(matplotlib.__file__)"
/opt/homebrew/Caskroom/miniconda/base/lib/python3.12/site-packages/matplotlib/__init__.py
anna@laptop project $ uv run python -c "import matplotlib; print(matplotlib.__file__)"
Traceback (most recent call last):
  File "<string>", line 1, in <module>
    import matplotlib; print(matplotlib.__file__)
    ^^^^^^^^^^^^^^^^^
ModuleNotFoundError: No module named 'matplotlib'
anna@laptop project $
```

Two paths. The first is whatever *my* shell found; the second is *this project's* interpreter. Then the same import, asked of each.

What you just saw, on the map



Part 2 — **uv**: the one workflow

The four commands

```
uv sync           # install everything the project declares
uv add pandas     # declare and install a new dependency
uv run python script.py  # run a script inside the project's environment
uv run jupyter lab # launch JupyterLab inside the project's environment
```

Typed in your terminal, always from inside the project folder. That is the whole workflow for this program.

Watch this one; nothing to type.

pyproject.toml and uv.lock, in plain words

```
pyproject.toml

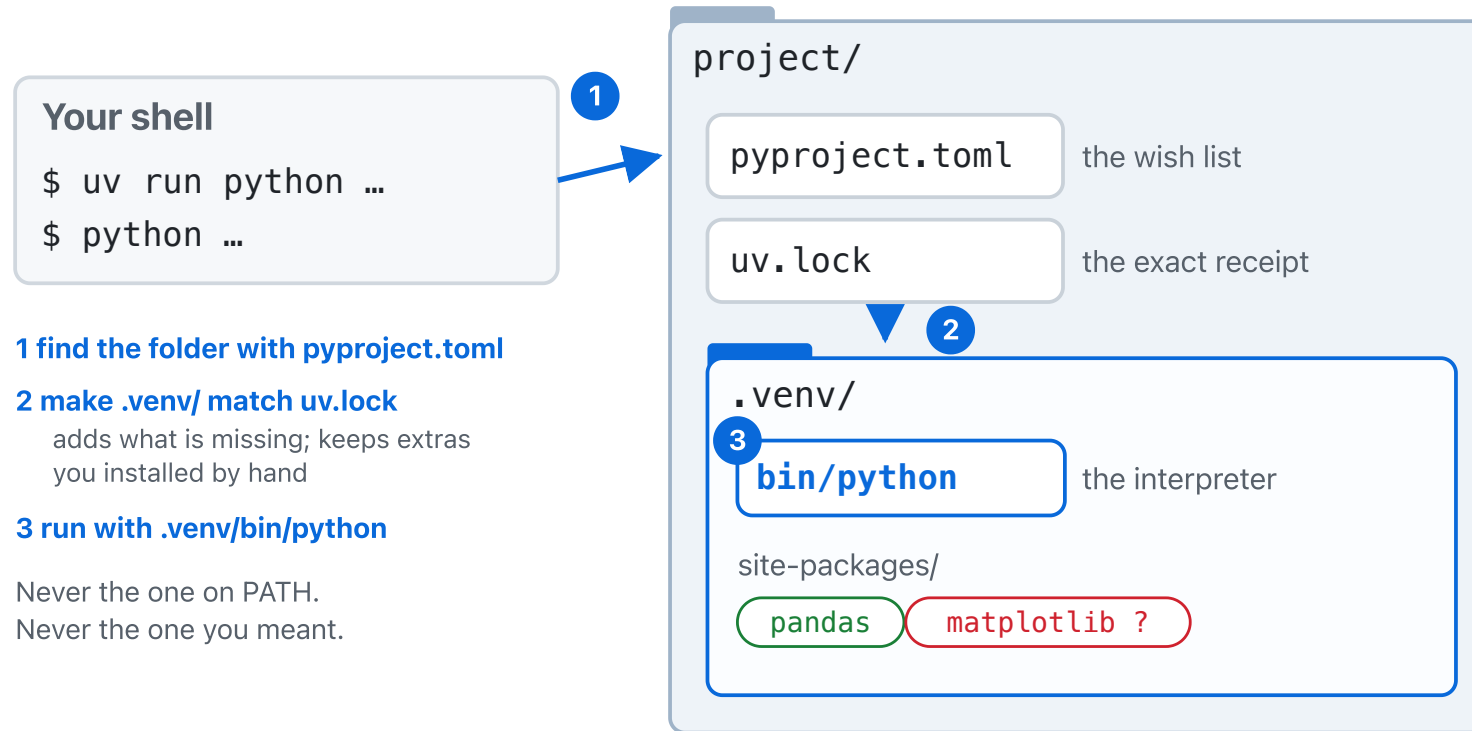
[project]
name = "ecbs5293-lab03-environments"
version = "0.1.0"
description = "ECBS5293 lab 3: it imports on ..."
requires-python = ">=3.10"
1 dependencies = [
    "pandas",
]
```

```
uv.lock (one of its entries)

[[package]]
2 name = "pandas"
  version = "3.0.5"
  source = { registry = "https://pypi.org/simple" }
3 dependencies = [
  sdist = { ..., hash = "sha256:dca3734d6ab7c906e6730f..." }
```

- 1 The wish list: what this project needs, by name. uv add writes the line for you; you do not hand-edit it.
- 2 The receipt: the exact version that satisfied the wish, for every package, including ones you never named.
- 3 A hash of the bytes: uv sync installs the same thing on every machine, and refuses anything else.

So what was `uv run` doing?



Other Pythons on this laptop

python.org 3.14

its own site-packages:

`numpy`

Microsoft Store

its own site-packages:

(whatever was installed there)

Anaconda base

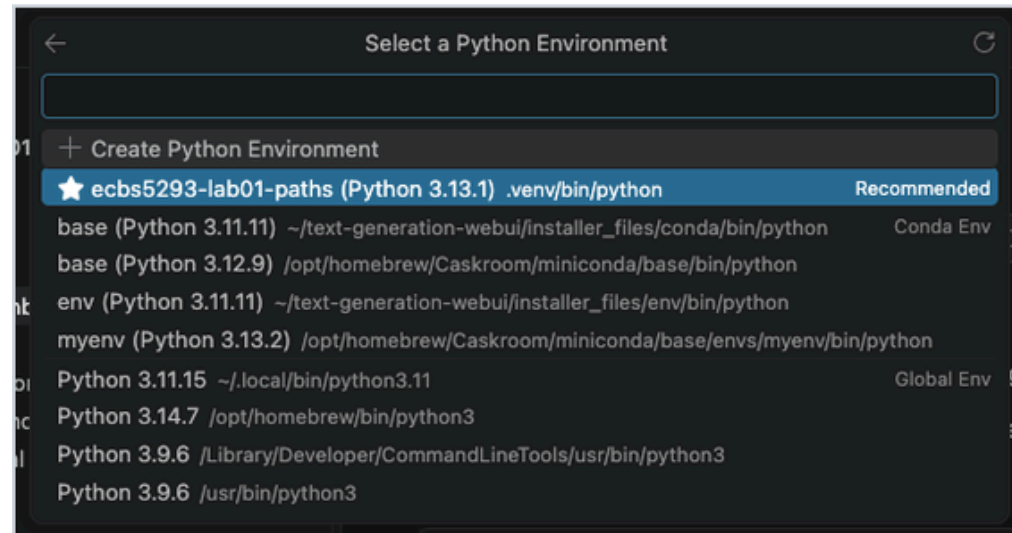
its own site-packages:

`pandas`

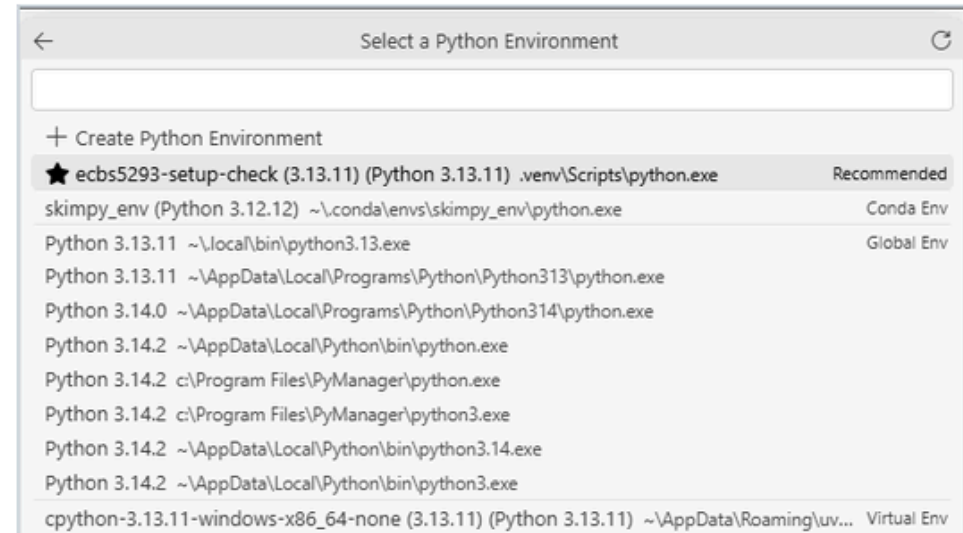
`matplotlib`

The notebook standard: the project's own kernel

macOS: `.venv/bin/python`



Windows: `.venv\Scripts\python.exe`



1. Open the **project folder** (never the lone `.ipynb`).
2. Kernel picker → the interpreter inside `.venv/`.
3. Verify: `import sys; sys.executable` in a cell prints a path inside *this project's* `.venv/`.

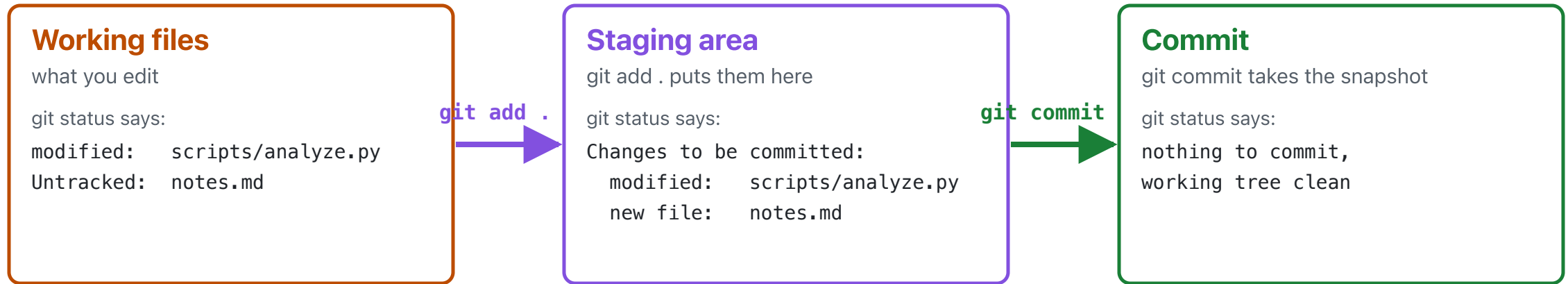
Alternative: `uv run jupyter lab` from the project folder offers the project's kernel by default. Verify it the same way.

In the wild: `requirements.txt`

- Older repos, blog posts, and other courses ship `requirements.txt` instead of `pyproject.toml`
- Same job, older file: *make this project's packages available to its interpreter*
- `uv` handles it: `uv pip install -r requirements.txt`
- **Not lectured.** The Lab 3 stretch task and the reference in the README cover it.

If a README says `pip install -r requirements.txt` and there is no such file, that README is wrong.

A whiff of Git — save a checkpoint



git diff compares only these two. Staged work and new files are invisible to it.

```
git status          # what changed?
git add .           # stage it: "include these in the next snapshot"
git commit -m "Fix the output path in analyze.py; add notes"
```

A commit is a **checkpoint after a working fix**. Commit when it works. Not before, not "at the end". **Watch now; you type these in the lab.**

After you commit: `git status` first, then `git diff`

```
anna@laptop project $ git status --short
M scripts/analyze.py
?? notes.md
anna@laptop project $ git add .
anna@laptop project $ git diff
anna@laptop project $ git status --short
A notes.md
M scripts/analyze.py
anna@laptop project $ git commit -q -m "Fix the output path in analyze.py; add notes"
anna@laptop project $ git status
On branch main
nothing to commit, working tree clean
anna@laptop project $
```

- `git status` covers everything: unstaged edits, staged work, and files Git has never seen. "nothing to commit, working tree clean" is the completeness check.
- `git diff` alone shows **unstaged edits to tracked files**. Staged work and new files are invisible to it.

AI thread — ask for evidence, not for a fix

Good questions for an assistant today:

- *"What evidence distinguishes 'package not installed' from 'installed in a different environment'?"*
- *"What does `uv sync` actually do? What is `pyproject.toml` ? How is `uv` different from `pip` or `conda` ?"*
- *"Here is `sys.executable` from my terminal and from my notebook. They differ. What does that mean?"*

Then run the inspection yourself and paste the real output.

The active interpreter decides what is true

- AI can suggest causes. It cannot see your machine.
- `sys.executable` shows which interpreter answered; printing `sys.path` shows where it searched.
- If the assistant says "it's installed" and the interpreter says `ModuleNotFoundError`, the interpreter is right.

Sources of truth today: the filesystem, the interpreter, `uv` 's output. Everything else is a hypothesis.

5-minute stand-and-stretch

Stand up. Leave the laptop. Back in five.

Lab 3 — It imports on my machine, but not here

First 15 minutes: no AI. Chat windows and assistant panels closed; VS Code itself stays open. Terminal, filesystem, and `uv` output only. Search engines are fine.

Why: every failure today is one an assistant diagnoses instantly. The reflex we want is *"check which interpreter,"* not *"ask what to check."* The exam has no chat window. Solve it in five minutes and write the note; the fifteen minutes are for looking, not for suffering.

TAs will ask you to close a window. That is not a penalty.

The symptom, and how to run it

```
anna@laptop ecbs5293-lab03-environments $ uv run python scripts/plot.py
Traceback (most recent call last):
  File "/Users/Shared/anna/ecbs5293-lab03-environments/scripts/plot.py", line 8, in <module>
    import matplotlib
ModuleNotFoundError: No module named 'matplotlib'
anna@laptop ecbs5293-lab03-environments $
```

In your terminal, in the Lab 3 folder you cloned at the start of the block:

```
uv sync
uv run python scripts/plot.py
```

The README says `pip install -r requirements.txt`. There is no such file. Your job: which interpreter is looking, what the project *declares*, and why they disagree. Edit files in VS Code (open the folder; trust it if asked). At home: the README begins with the `git clone` line.

What you produce, and the last ten minutes

1. After a plain `uv sync`, `uv run python scripts/plot.py` runs clean and writes the PNG. Sync first, then run.
2. `pyproject.toml` declares the missing dependency (`uv add`, not a hand edit)
3. README setup instructions are correct for *this* repo
4. One commit of the working state; `git status` reports a clean tree afterward
5. `DIAGNOSIS.md` : symptom, cause, evidence (paste real output), change, verification
6. At minute 33: explain it to a neighbour, about a minute, no notes; the what-if about environments is on the next slide. Checkpoint on Moodle before you leave.

Stretch: the `requirements.txt` reference in the README. Reset instructions are in the README; both resets destroy work.

The last ten minutes — explain it to a neighbour

Speaker, about a minute, pointing at the screen, not reading the note: symptom → cause → evidence → change → verification.

Listener asks:

1. Show me the evidence.
2. Why did it fail — not just where?
3. Suppose you had fixed this with `uv pip install matplotlib` instead of `uv add`, and committed. A classmate clones your repo and runs `uv sync && uv run python scripts/plot.py`. What happens, and why?

Swap. Unsure, or you disagree? **Hand up.** Then the answer, for everyone. Before you leave: `DIAGNOSIS.md` to the Lab 3 checkpoint on Moodle.